Lottery Ticket Hypothesis

Rewinding y alcance de la poda en MLP, CNN, ResNet y ViT

Deep Learning y Sistemas Inteligentes - CC3092
Proyecto Final

José Antonio Mérida Castejón

1 de noviembre de 2026

Resumen

La *Lottery Ticket Hypothesis* (LTH) afirma que dentro de una red neuronal grande existe una subred pequeña que, entrenada desde sus mismos pesos iniciales, alcanza una accuracy comparable a la de la red completa. Este proyecto la reproduce y la pone a prueba en cuatro arquitecturas de complejidad creciente (MLP, CNN, ResNet de 20 capas y un ViT pequeño, la que se estudió con más profundidad) con tres preguntas: (1) ¿existen *winning tickets* en cada arquitectura?, (2) ¿a medida que la red es más compleja hace falta rebobinar (*rewinding*) los pesos a un punto posterior del entrenamiento, y qué papel puede jugar el *warmup*?, y (3) ¿importa podar por capa o globalmente? Se encontraron tickets en las cuatro arquitecturas (en la ResNet solo con rewinding); la necesidad de rebobinar es compatible con la complejidad de la red, pero está confundida con el *warmup* y el optimizador, por lo que no se puede atribuir a la arquitectura; y el tipo de poda cambió el resultado de la CNN, el caso donde se comparó directamente: con poda por capa el ticket colapsa con 10.7 % de los pesos (53.0 % frente a 61.2 % del control aleatorio) y con poda global no (75.7 % frente a 69.4 %), porque la poda por capa obliga a las capas pequeñas, cuyos pesos son los más grandes, a perder tanto como las grandes. Que las redes más grandes necesiten poda global es consistente con esto pero no se probó. Todos los experimentos son de una sola semilla.

Por completar

Datos del curso (universidad, catedrático, fecha de entrega), integrantes del grupo, enlace al repositorio público y enlace al video. La instrucción pide repositorio público, video del funcionamiento y presentación de máximo 10 diapositivas.

Índice

1. Descripción del problema **3**
- 1.1. Contexto . . . 3
- 1.2. Problema . . . 3
- 1.3. Hipótesis . . . 3
- 1.4. Objetivos . . . 4

2. Análisis **4**
- 2.1. Poda iterativa por magnitud y definición de ticket . . . 4
- 2.2. Qué se espera ver y cómo se decide . . . 4
- 2.3. Alcance de la poda: por qué podría importar . . . 4
- 2.4. Rewinding y warmup . . . 5
- 2.5. Trabajo relacionado y límites de la comparación . . . 5

3. Propuesta de solución **5**

4. Descripción de la solución **6**
- 4.1. Arquitecturas . . . 6
- 4.2. Implementación . . . 7
- 4.3. Hiperparámetros . . . 7
- 4.4. Reproducibilidad y costo . . . 7

5. Herramientas aplicadas **8**
- 5.1. Herramientas . . . 8
- 5.2. Uso de IA generativa: prompts y evolución de la idea . . . 8

6. Resultados **9**
- 6.1. H0: existencia de tickets . . . 9
- 6.2. ViT con entrenamiento largo: ¿era un mal baseline? . . . 11
- 6.3. H1: rewinding según la arquitectura . . . 12
- 6.4. H2: ¿hace falta warmup? (observacional) . . . 12
- 6.5. H3: poda por capa frente a poda global (CNN) . . . 13
- 6.6. Inspección de los tickets del ViT largo . . . 16
- 6.7. Incertidumbre . . . 22

7. Discusión **22**
- 7.1. Qué dicen los resultados sobre cada hipótesis . . . 22
- 7.2. Limitaciones . . . 23
- 7.3. Preguntas de discusión generadas con IA . . . 23

8. Conclusión **24**
- 8.1. Trabajo futuro . . . 25

A. Desviaciones respecto a las fuentes **26**

B. Cómo reproducir **26**

1. Descripción del problema

1.1. Contexto

Las redes neuronales modernas tienen millones de parámetros, pero la mayoría resulta prescindible: tras el entrenamiento se pueden eliminar (*podar*) hasta el 90 % de los pesos de menor magnitud con poca pérdida de accuracy. Eso reduce memoria y cómputo en inferencia y tiene aplicación directa en la industria (modelos para dispositivos móviles y embebidos, menor costo energético) y en la academia (qué información realmente usa una red). Sin embargo, la poda tradicional se aplica a una red *ya entrenada*: no reduce el costo de entrenarla. Frankle y Carbin [1] mostraron que existe una alternativa: si la subred podada se reentrena *desde la misma inicialización* de la red original, alcanza una accuracy igual o mayor. A esa subred la llaman *winning ticket*.

1.2. Problema

El resultado original se obtuvo en redes pequeñas (LeNet-300-100, Conv-2/4/6). En redes más profundas, regresar los pesos a la inicialización deja de funcionar con el *learning rate* estándar, y es necesario rebobinar a una iteración temprana del entrenamiento [2]. Esto plantea la pregunta central del proyecto:

> *¿Cómo cambia la existencia de los tickets, y lo que hay que hacer para encontrarlos, al pasar de arquitecturas simples a arquitecturas más complejas (MLP, CNN, ResNet, ViT)? ¿Y qué decisiones del procedimiento de poda cambian ese resultado?*

1.3. Hipótesis

Se formulan cuatro hipótesis, de las que el proyecto prueba unas directamente y otras solo de forma observacional:

H0 – Existencia.
En cada arquitectura existe una subred con pocos pesos que, desde su inicialización (o desde un punto de rewind), iguala o supera a la red densa y supera a la *misma máscara con pesos nuevos* (control aleatorio). *Prueba directa:* las curvas de accuracy contra pesos restantes.

H1 – Complejidad y rewinding.
Mientras más compleja es la red, más tarde en el entrenamiento aparece el ticket: el MLP y la CNN no necesitan rebobinar, la ResNet sí, y el ViT también. *Prueba parcial:* se compara ticket desde θ_0 contra ticket con rewinding en cada arquitectura, pero con uno o dos puntos de rewind por arquitectura; no se barre el punto de rewind k.

H2 – Warmup.
El *warmup* del *learning rate* cumple una función parecida al rewinding (evitar el arranque inestable), por lo que una arquitectura con warmup necesita rebobinar menos. *Prueba observacional, no directa:* la ResNet se entrenó sin warmup y el ViT con warmup, pero el warmup no se manipuló de forma controlada. Está confundido con el optimizador (SGD frente a AdamW), los datos y el tamaño de la red.

H3a – Alcance de la poda (CNN).
La poda *global* (un solo umbral de magnitud para toda la red) funciona mejor que la poda *por capa* (20 % de cada capa) porque no obliga a las capas pequeñas a perder la misma fracción que las grandes. *Prueba directa:* el experimento 02b repite el 02 cambiando únicamente el alcance de la poda.

H3b – Redes más grandes necesitan poda global.
A medida que la red crece (más capas, y de tamaños muy distintos), la poda por capa se vuelve más dañina porque las capas pequeñas se convierten en un cuello de botella. *No probada:* ResNet y ViT se podaron solo de forma global; la CNN es el único caso con ambas variantes.

1.4. Objetivos

1. Implementar poda iterativa por magnitud (IMP) con rewinding y control aleatorio, reutilizable en las cuatro arquitecturas.
2. Medir accuracy contra porcentaje de pesos restantes en cada arquitectura y para cada variante (ticket, ticket con rewinding, aleatorio).
3. Inspeccionar *qué* encontró la poda (pesos vivos por capa, máscaras, atención, gradientes), no solo cuánta accuracy queda.
4. Documentar con honestidad qué se puede y qué no se puede concluir con los datos obtenidos.

2. Análisis

2.1. Poda iterativa por magnitud y definición de ticket

IMP repite el siguiente ciclo (Figura 1, parte inferior): (1) se entrena la red con la máscara actual; (2) se elimina una fracción p de los pesos vivos de menor magnitud $|w|$; (3) los pesos sobrevivientes se regresan a un valor de referencia; (4) se vuelve a entrenar. Con $p = 20\,\%$, tras n rondas quedan $0{,}8^n$ de los pesos (10.7 % tras 10 rondas); con $p = 35\,\%$ en el ViT, $0{,}65^n$.

El valor de referencia define el método: la inicialización θ_0 (ticket clásico [1]) o el valor tras k épocas de entrenamiento denso θ_k (rewinding [2]). Para saber si el resultado es mérito de la *estructura* o de los *valores iniciales*, se usa un control: la misma máscara con pesos reinicializados al azar. Si el ticket supera al control, los valores iniciales importan.

2.2. Qué se espera ver y cómo se decide

- **Existencia (H0):** la curva del ticket se mantiene a menos de 1 punto de la red densa o por encima, y por encima de la curva aleatoria con un margen mayor al ruido.
- **Rewinding (H1):** si el ticket desde θ_0 no supera al control pero el ticket rebobinado sí, hace falta rebobinar.
- **Alcance de la poda (H3):** si la caída del ticket a poca densidad desaparece al cambiar de poda por capa a poda global con todo lo demás igual.

2.3. Alcance de la poda: por qué podría importar

En la poda *por capa* cada capa pierde exactamente la misma fracción. Las capas pequeñas (como la primera convolución, con 1,728 pesos) quedan con muy pocos pesos en términos absolutos, mientras que las capas grandes (la primera capa lineal de la CNN tiene 2.1 M pesos, el 86 % del total) pierden en proporción lo mismo aunque sean redundantes. La poda *global* compara las magnitudes de todos los pesos con un único umbral, de modo que cada capa conserva lo que su magnitud justifica. El riesgo opuesto es que una capa se quede sin pesos (*layer collapse*); aquí se mide con filtros y neuronas completamente vacíos.

2.4. Rewinding y warmup

Frankle et al. [2] muestran que en ResNet-18 y VGG-19 con *learning rate* 0.1 el ticket desde θ_0 no es mejor que la reinicialización, pero rebobinar a una iteración temprana (cientos de iteraciones) sí lo es. Frankle y Carbin [1] (apéndice) reportan que en esas redes aparecen tickets con un *learning rate* menor o con warmup largo. Esto sugiere que el rewinding y el warmup atacan el mismo problema: el arranque del entrenamiento con un *learning rate* alto es inestable y los pesos iniciales aún no están en una región donde la máscara sea estable. Esta es la base de H2; en este proyecto no se manipuló el warmup, solo se observó su coincidencia con la necesidad de rewinding en cada arquitectura (Sección 6.4).

2.5. Trabajo relacionado y límites de la comparación

Para los ViT, Shen et al. [3] reportan que los tickets a nivel de peso son difíciles de encontrar en DeiT-Small con ImageNet; el ViT de este proyecto es pequeño y se entrena en CIFAR-10, así que sus resultados no son comparables directamente. Renda et al. [4] comparan el rewinding con el ajuste fino tras la poda; aquí solo se toma su receta (SGD con *weight decay* en ResNets de CIFAR-10) como referencia. El recipe de entrenamiento del ViT sigue las pautas de DeiT [5] (AdamW, *weight decay* 0.05, warmup), con las desviaciones del Apéndice A.

3. Propuesta de solución

Para contestar las preguntas se diseñó una serie de experimentos que comparten el mismo código de poda (`lth.py`) y varían la arquitectura, el punto de rewind y el alcance de la poda. Cada experimento mide las tres curvas de la Sección 2.1 (ticket, ticket con rewinding, control aleatorio) contra el porcentaje de pesos restantes.

Exp.	Arquitectura / datos	Poda	Rewind	Hipótesis que prueba	Epochs/ronda
01	MLP LeNet-300-100 / MNIST	capa	–	H0	5
02	CNN Conv-4 / CIFAR-10	capa	–	H0, H3 (línea base)	30
02b	CNN Conv-4 / CIFAR-10	**global**	–	**H3**	30
03	ResNet de 20 capas / CIFAR-10	global	época 1	H0, H1, (H2)	77
04	ViT pequeño / CIFAR-10	global	época 2	H0, H1, (H2)	20
05	ViT, entrenamiento largo	global	época 5	H0, H1, (H2); validar el baseline	100
06	Inspección de los checkpoints de 05	–	–	qué encontró la poda	–

Cuadro 1: Matriz de experimentos. (H2) indica que el warmup solo se observa, no se manipula.

Enfoque del estudio. El proyecto no estudia las cuatro arquitecturas con la misma profundidad. El **ViT** es la arquitectura a la que se dedicó más trabajo: dos corridas (20 y 100 épocas) con las tres curvas, validación de su baseline y la inspección completa de los checkpoints (pesos vivos por capa, accuracy por clase, atención, entropía y gradientes). La **CNN** es el caso donde se comparó directamente la poda por capa y la global, con inspección de máscaras. El MLP y la ResNet sirven como puntos de referencia (una corrida cada uno, la ResNet con rewinding).

Cambios deliberados entre experimentos. 02b cambia *solo* el alcance de la poda respecto a 02. 05 repite 04 con 100 épocas, más aumento de datos y *label smoothing* porque el baseline de 04 (78.9 %) parecía subentrenado y eso podría explicar que el ticket superara a la red densa. El ViT se mide dos veces (04 y 05) para separar el efecto de la poda del de un mal baseline.

Criterios de calidad. Cada curva se compara con la accuracy de su red densa y con el control aleatorio; la inspección de las máscaras y de las redes sirve para explicar las diferencias, no para reemplazar la medición. Se declara explícitamente que una sola semilla y 10,000 imágenes de prueba permiten distinguir diferencias de unos 5 puntos, pero no de 1 punto o menos.

4. Descripción de la solución

4.1. Arquitecturas

La Figura 1 muestra las cuatro redes y el ciclo de IMP.

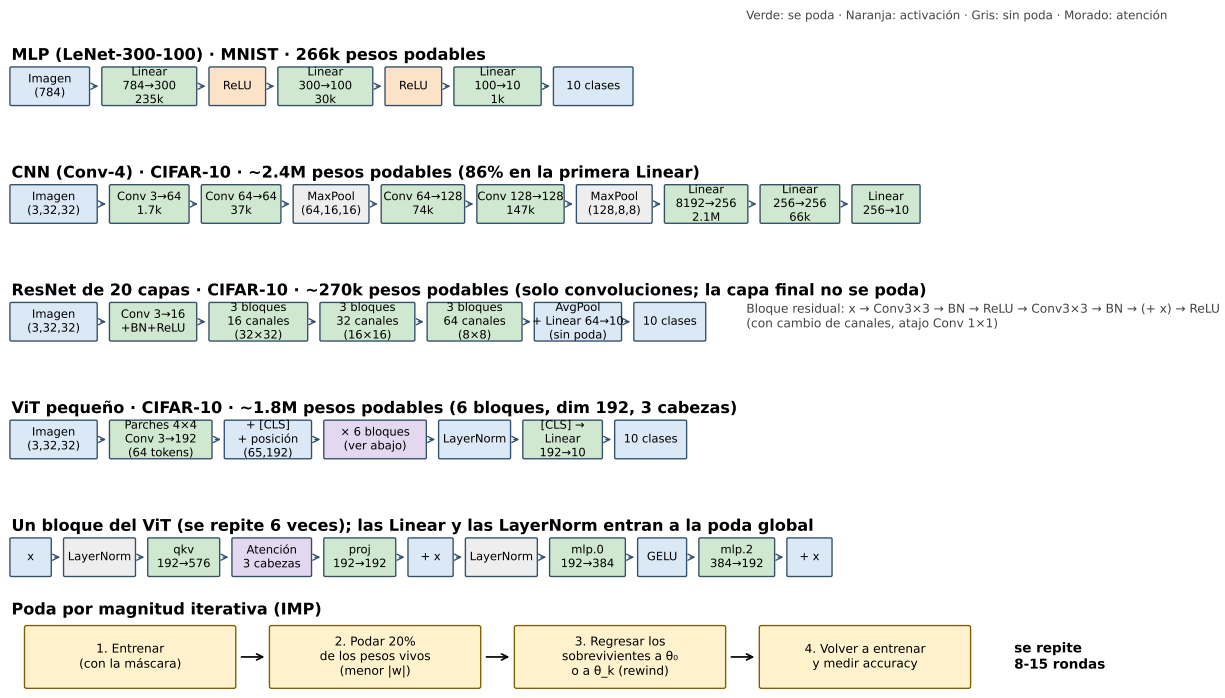


Figura 1: Arquitecturas usadas (forma de los tensores y pesos podables por capa) y ciclo de IMP.

MLP (LeNet-300-100). $784 \to 300 \to 100 \to 10$ con ReLU; 266k pesos podables.

CNN (Conv-4). Cuatro convoluciones 3×3 (64, 64, 128, 128 filtros) con *max-pooling* y tres capas lineales; 2.4M pesos, el 86 % en la primera capa lineal ($8192 \to 256$).

ResNet de 20 capas. Arquitectura del apéndice I.3 de [1]: convolución inicial más 9 bloques residuales en tres etapas de 16, 32 y 64 canales, con atajos convolucionales 1×1 cuando cambian las dimensiones; 270k pesos convolucionales.

ViT pequeño. Parches de 4×4 (64 tokens más `[CLS]`), dimensión 192, 6 bloques de atención con 3 cabezas y MLP de 384; 1.8M pesos podables. Todas las capas lineales y las LayerNorm entran en la poda global.

4.2. Implementación

Los cuatro modelos, los ciclos de entrenamiento y evaluación y el módulo de poda `lth.py` fueron escritos por el autor en PyTorch; el guardado de checkpoints y la mayor parte de los cuadernos de experimentos los implementó la IA (Sección 5.2). El módulo `lth.py` contiene:

- `run_imp`: bucle de IMP con alcance `"layer"` o `"global"`, ritmo de poda configurable, rewind a la época k y guardado de cada ronda;
- `run_lth`: ejecuta las tres curvas (ticket, ticket con rewinding, control aleatorio con la misma máscara y otra inicialización);
- `save_base`, `save_round` / `list_checkpoints` (implementados con IA): guardan pesos y máscaras de cada ronda para poder inspeccionarlos después sin reentrenar.

Los experimentos son cuadernos `marimo` guardados como `.py` (reactivos y versionables), exportables a HTML. La inspección (06), la comparación (05) y el reporte (07) leen los resultados guardados; la demo interactiva (08) usa los checkpoints para comparar modelos densos, tickets y controles sobre una imagen elegida.

4.3. Hiperparámetros

	MLP	CNN	ResNet	ViT (04)	ViT largo (05)
Optimizador	Adam 1.2e-3	Adam 3e-4	SGD 0.1, mom. 0.9	AdamW 1e-3, wd 0.05	igual
Aumento de datos	no	no	flip + crop	flip + crop	+ RandAugment, erasing, mixup/cutmix, LS
Poda por ronda	20 % capa	20 % capa / global	20 % global (solo conv.)	35 % global	35 % global
Rondas	15	10	10	8	8
Warmup	no	no	no	2 ép. + coseno	5 ép. + coseno

Cuadro 2: Configuración de cada experimento (LS = *label smoothing*). Las desviaciones respecto a las fuentes están en el Apéndice A.

4.4. Reproducibilidad y costo

Entorno gestionado con `uv` (`uv.lock`); una sola semilla por experimento y entrenamiento en GPU (no determinista). Todo el proyecto usó unas 10 horas de GPU (RTX 4070 SUPER, 4 kWh). Las figuras y tablas de este informe se generan con `report/make_figures.py` a partir de `results/` y `checkpoints/`.

5. Herramientas aplicadas

5.1. Herramientas

- **PyTorch / torchvision**: modelos, entrenamiento y datasets (MNIST, CIFAR-10).
- **marimo**: cuadernos reactivos guardados como `.py`, exportables a HTML. Es una herramienta nueva para este curso (a diferencia de Jupyter, el orden de ejecución lo determina el grafo de dependencias entre celdas y el archivo es código normal, apto para git).
- **uv**: gestor de entornos reproducibles de Python.
- **Claude Code**: asistente de IA en la terminal, usado para leer el repositorio, ejecutar corridas, generar figuras de inspección y redactar borradores de análisis.
- **matplotlib**: figuras (el código de las figuras de inspección fue generado con ayuda de IA).
- **LaTeX**: este informe; las figuras y tablas numéricas se generan por script.

5.2. Uso de IA generativa: prompts y evolución de la idea

Por completar

Esta sección es obligatoria según la instrucción del proyecto. Falta el **historial de prompts más relevantes**, indicando para cada uno si el resultado fue satisfactorio o presentó errores, por qué se cree que ocurrió, y cómo se evaluó la respuesta. Abajo hay la estructura y los prompts de los que ya hay registro; completar con los de las sesiones anteriores.

Reparto del trabajo. Del autor: el tema, la pregunta y las hipótesis; los cuatro modelos (`nn.Module`); los ciclos de entrenamiento y evaluación; el código de poda `lth.py` (máscaras, IMP, control aleatorio); y ejecutar y verificar los experimentos. **De la IA (Claude, mediante Claude Code):** el guardado de pesos y máscaras (checkpoints); la implementación de la mayor parte de los cuadernos de experimentos, cuyas partes centrales el autor sabía correctas y revisó; las visualizaciones, los diagramas de arquitectura y las tablas automáticas; la revisión de código e hiperparámetros contra las fuentes; borradores de interpretación y las preguntas de discusión de la Sección 7.3 (marcadas como generadas con IA); y la estructura del informe y de la presentación.

Prompts de los que hay registro.

- *"Can these models be exported so I can run experiments on them? Using masks to see how they turn out."* → origen del guardado de checkpoints (`save_round`) y del cuaderno de inspección.
- *"I wanna handpick from results to see what changes... also saving % of original params."* → checkpoints por ronda con el porcentaje de pesos en el nombre.
- *"ViT longer has to have more epochs + maybe more aggressive data augmentation."* → experimento 05.

Por completar

Tabla del historial de prompts (prompt, qué se obtuvo, ¿satisfactorio?, por qué, cómo se validó). Dos ejemplos concretos del proyecto que se pueden desarrollar: (1) la primera corrida del ViT denso largo se descartó y repitió; (2) el reporte afirmaba una explicación causal del colapso de la CNN que solo se pudo confirmar al correr 02b, y por eso se redactó como *posible, no probada* hasta entonces.

Prompt (resumen)	Resultado	Evaluación y por qué
[completar]	*[completar]*	*[completar]*
[completar]	*[completar]*	*[completar]*

Cuadro 3: Historial de prompts relevantes (plantilla).

Evaluación crítica de la IA. Todo número que aparece en el informe viene de `results/*.json` o de los checkpoints; los textos de interpretación redactados con ayuda de IA se marcan como *borrador* y se contrastan con los datos antes de aceptarlos. Se identificaron como riesgos: explicaciones causales presentadas con más certeza que la que los datos permiten (por ejemplo, atribuir resultados a la regularización o al warmup sin experimento que los separe) y figuras que se ven bien sin que demuestren lo que parecen demostrar (los mapas de atención). Por ello cada afirmación causal del informe indica qué experimento la probaría.

6. Resultados

6.1. H0: existencia de tickets

La Figura 2 muestra las curvas de todas las corridas y la Tabla 4 las resume. Las tres curvas son: ticket desde θ_0 (azul), ticket con rewinding (verde) y reinicialización aleatoria de la misma máscara (naranja punteada).

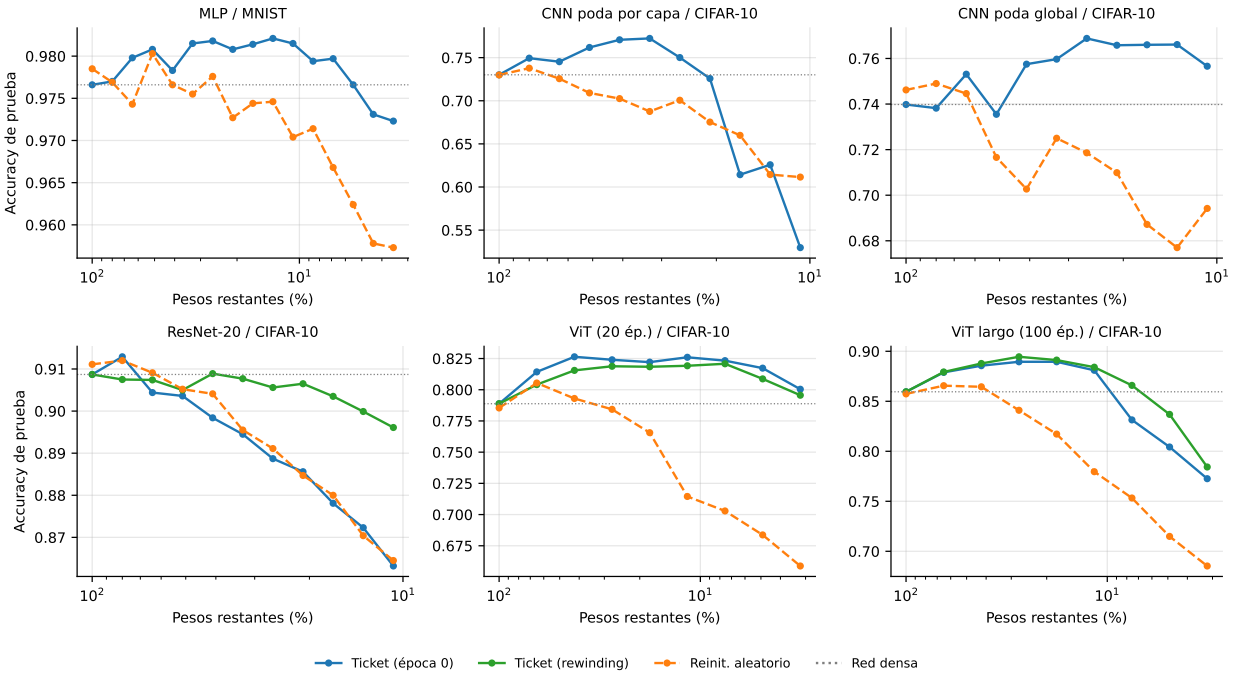


Figura 2: Accuracy de prueba contra porcentaje de pesos restantes (eje x logarítmico e invertido: la red se hace más pequeña hacia la derecha).

Arquitectura	Densa (%)	Mín. pesos ticket	Mín. pesos aleat.	Ventaja ticket (pts)	Ganancia rewind (pts)
MLP / MNIST	97.7	3.5 %	6.9 %	+0.8	–
CNN poda por capa / CIFAR-10	73.0	21.0 %	64.0 %	+2.2	–
CNN poda global / CIFAR-10	74.0	10.7 %	64.0 %	+4.4	–
ResNet-20 / CIFAR-10	90.9	51.2 %	41.0 %	-0.1	+1.5
ViT (20 ép.) / CIFAR-10	78.9	3.2 %	27.5 %	+8.1	-0.7
ViT largo (100 ép.) / CIFAR-10	86.0	11.6 %	42.3 %	+6.4	+1.1

Cuadro 4: Resumen. "Mín. pesos": la red más pequeña a menos de 1 punto de la accuracy densa. "Ventaja del ticket": diferencia promedio entre el ticket y el control aleatorio sobre las rondas podadas. "Ganancia del rewind": diferencia promedio entre el ticket con rewinding y el clásico.

MLP / MNIST. El ticket se mantiene entre 97.2 % y 98.2 % hasta 3.5 % de los pesos, a la par de la red densa (97.7 %), mientras que la reinicialización aleatoria cae a 95.7 %. No se probó el rewinding. En la primera capa, 128 de los 784 píxeles de entrada quedan sin ningún peso (el borde de la imagen), lo cual es esperable porque los píxeles que siempre valen 0 no reciben gradiente.

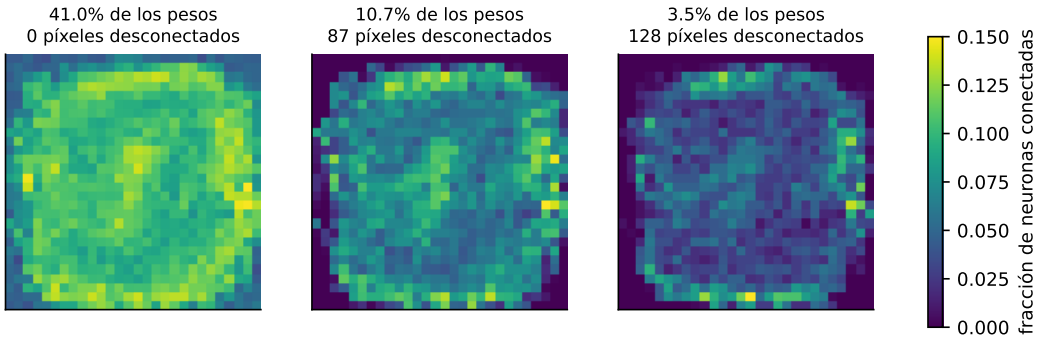


Figura 3: MLP: para cada píxel de la imagen de 28×28, la fracción de las 300 neuronas ocultas que sigue conectada a él en la primera capa, en tres niveles de poda.

Cómo leerla: oscuro = el píxel casi no se usa. Con 41 % de los pesos todavía todos los píxeles tienen alguna conexión; con 3.5 %, 128 píxeles del borde (donde las dígitos de MNIST casi siempre valen 0) quedan completamente desconectados y el resto se concentra en el centro de la imagen.

CNN / CIFAR-10. Con poda por capa el ticket supera a la red densa (73.0 %) hasta con 26 % de los pesos, con un máximo de 77.2 % con 33–41 %, y supera al control hasta con 21 %; por debajo de eso cae a 61.4 % con 16.8 %, 62.6 % con 13.4 % y 53.0 % con 10.7 %, por debajo del control en 16.8 % y 10.7 % (ver Sección 6.5). Con poda global el ticket se mantiene entre 73.6 % y 76.9 %, a la par o por encima de la red densa (74.0 %) en todo el rango y por encima del control desde 64 % de los pesos.

ResNet / CIFAR-10. Con *learning rate* 0.1 el ticket desde θ_0 **no** supera al control (86.3 % frente a 86.5 % con 10.7 % de los pesos), igual que en [2]. Rebobinar a la época 1 recupera casi toda la accuracy: 89.6 % con 10.7 % de los pesos (red densa: 90.9 %).

ViT / CIFAR-10. Con 20 épocas el ticket queda en 82–83 % de 42 % a 7.5 % de los pesos, por encima de la red densa (78.9 %); el control cae a 65.9 % con 3.2 %. El baseline probablemente está

subentrenado, motivo del experimento 05.

6.2. ViT con entrenamiento largo: ¿era un mal baseline?

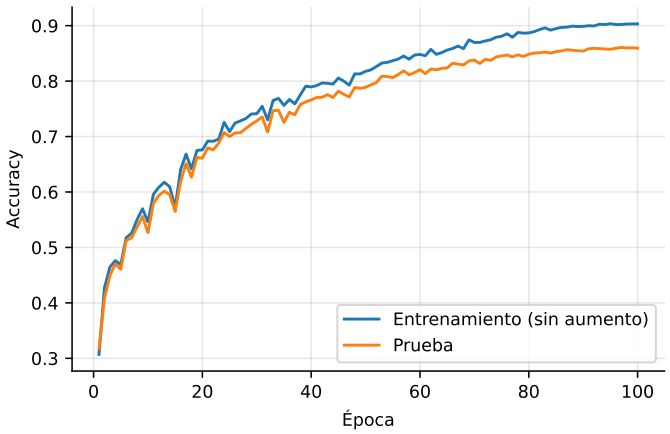


Figura 4: ViT denso de 100 épocas: accuracy de entrenamiento (sin aumento de datos) y de prueba por época.

Con 100 épocas la red densa llega a 86.0 % de accuracy de prueba (78.9 % en 04), con una brecha de unos 4 puntos respecto al entrenamiento y la prueba todavía subiendo: un baseline razonable. Medido otra vez desde otro arranque (ronda 0 del control aleatorio) da 85.7 %, así que la inicialización explica unos 0.2 puntos.

% de pesos	Ticket θ_0	Ticket rewind (ép. 5)	Aleatorio
100.0	86.0	86.0	85.7
65.0	87.9	87.9	86.6
42.3	88.6	88.8	86.5
27.5	88.9	89.5	84.1
17.9	88.9	89.1	81.7
11.6	88.1	88.4	78.0
7.5	83.1	86.6	75.3
4.9	80.4	83.7	71.5
3.2	77.3	78.4	68.5

Cuadro 5: ViT largo (05): accuracy de prueba (%) por porcentaje de pesos restantes.

El ticket sigue por encima de la red densa hasta ∼12 % de los pesos, así que el resultado de 04 no era solo un baseline débil. El ticket le gana al control aleatorio en todos los puntos, y la diferencia crece al podar más (de unos 5 puntos con 27.5 % de los pesos a unos 9 con 3.2 %).

6.3. H1: rewinding según la arquitectura

Arquitectura	Rewind probado	¿Hizo falta?	Evidencia
MLP	no	no	el ticket desde θ_0 iguala a la red densa hasta 3.5 % de los pesos
CNN	no	no	ídem (con poda global hasta 10.7 %)
ResNet	época 1	**sí**	desde θ_0 igual al aleatorio (86.3 vs. 86.5 %); con rewind 89.6 %
ViT 20 ép.	época 2	no	rewind queda $\approx$ ticket ($-0{,}7$ puntos en promedio)
ViT 100 ép.	época 5	solo con poca densidad	sin efecto con $\geq$12 % de pesos; +3.4 puntos con 7.5 % y +3.3 con 4.9 %, +1.2 con 3.2 %

Cuadro 6: Necesidad de rewinding por arquitectura.

Los datos son compatibles con la idea de que a más complejidad se necesita rebobinar (MLP y CNN no, ResNet sí), pero el ViT, la arquitectura más compleja, no lo necesita con densidades medias. La relación no es monótona en la complejidad; la explicación que más compite es H2 (Sección 6.4).

Borrador de interpretación (editar)

En el ViT largo el rewinding solo ayuda cuando queda poco: con 12 % o más de los pesos, la subred aún tiene capacidad para recuperarse desde cualquier inicialización razonable, y ticket, rewind y red densa quedan dentro de $\sim$1 punto, que es el ruido de una sola corrida. Con 7.5 % y 4.9 % la subred tiene poca capacidad y su entrenamiento es más sensible; Frankle et al. [2] encuentran que el ticket solo es estable si se arranca después de unas pocas iteraciones. Las 5 épocas del rewinding coinciden con el final del warmup, lo que es compatible con esa idea pero no se probó. En 04 el baseline subentrenado dejaba poco margen, y no tiene los puntos 7.5 % y 4.9 % donde 05 muestra la ganancia; las dos corridas no son comparables punto por punto. La ganancia de +3.4 puntos supera el ruido binomial ($\pm 0{,}6$), pero no se repitió con otra semilla.

6.4. H2: ¿hace falta warmup? (observacional)

Lo que *se observó*: la ResNet (sin warmup, *learning rate* 0.1 desde la época 0) necesita rewinding; el ViT (con warmup de 2 y 5 épocas) funciona desde θ_0 y solo aprovecha el rewind con muy poca densidad. Lo que *no se hizo*: variar el warmup dentro de una misma arquitectura. Las dos arquitecturas difieren además en el optimizador (SGD frente a AdamW), el tipo de capa, la normalización (BatchNorm frente a LayerNorm) y los datos aumentados. Por eso la conclusión permitida es: *el patrón es compatible con que el warmup haga el trabajo del rewinding [1, 2], pero no se puede separar de las otras diferencias.* La prueba directa sería repetir la ResNet con warmup (un ticket desde θ_0 debería aparecer) y el ViT sin warmup (el ticket desde θ_0 debería dejar de funcionar y el rewind ayudar).

Por completar

Decidir si este experimento (ResNet con warmup y ViT sin warmup, con una semilla cada uno: unas 1 h 45 min y 45 min de GPU) se corre antes de entregar. Si se corre, esta sección pasa de *observacional* a *directa*.

6.5. H3: poda por capa frente a poda global (CNN)

Con poda por capa el ticket de la CNN cae a partir de 16.8 % de los pesos y llega al mínimo con 10.7 %: 49.7 % frente a 61.0 % del control en la primera corrida, y 53.0 % frente a 61.2 % en la repetición (resultados de este informe), por lo que no es ruido. Con poda global (02b) el colapso desaparece: 75.7 % para el ticket frente a 69.4 % del control (red densa: 74.0 %). Pasa de 8 puntos *por debajo* del control a 6 puntos *por encima*.

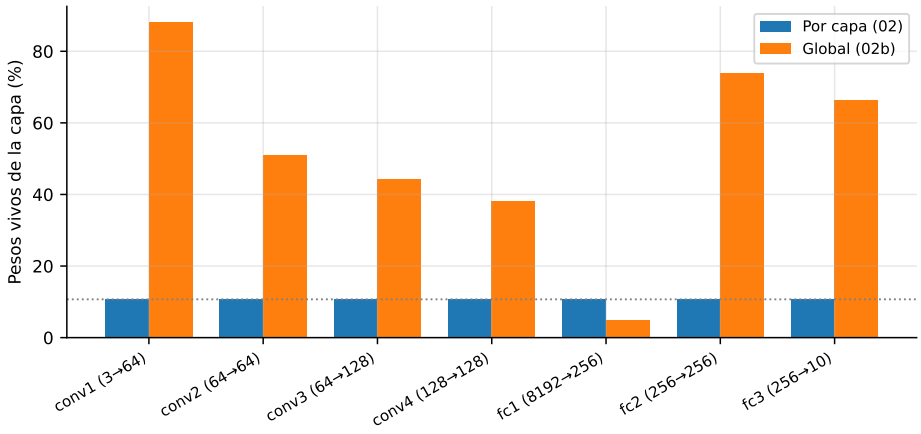


Figura 5: CNN: fracción de pesos vivos por capa en el ticket con 10.7 % de los pesos totales (línea punteada), poda por capa frente a poda global.

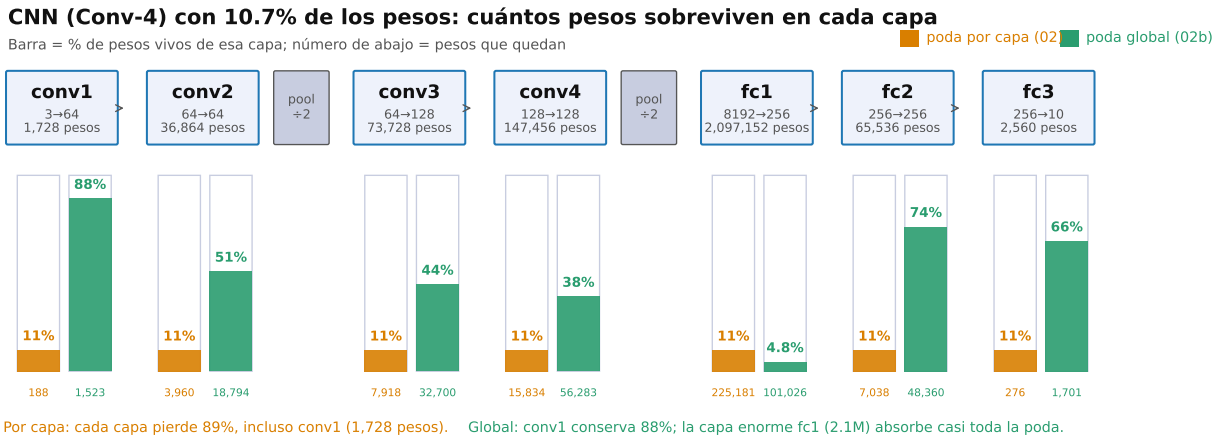


Figura 6: La CNN (Conv-4) con 10.7 % de los pesos totales. Sobre cada capa: forma y número de pesos; debajo: fracción de pesos vivos de esa capa con poda por capa (naranja) y poda global (verde), y número de pesos que quedan. Con poda por capa todas las capas quedan en ∼11 %; con poda global conv1 conserva 88 % y fc1 solo 4.8 %.

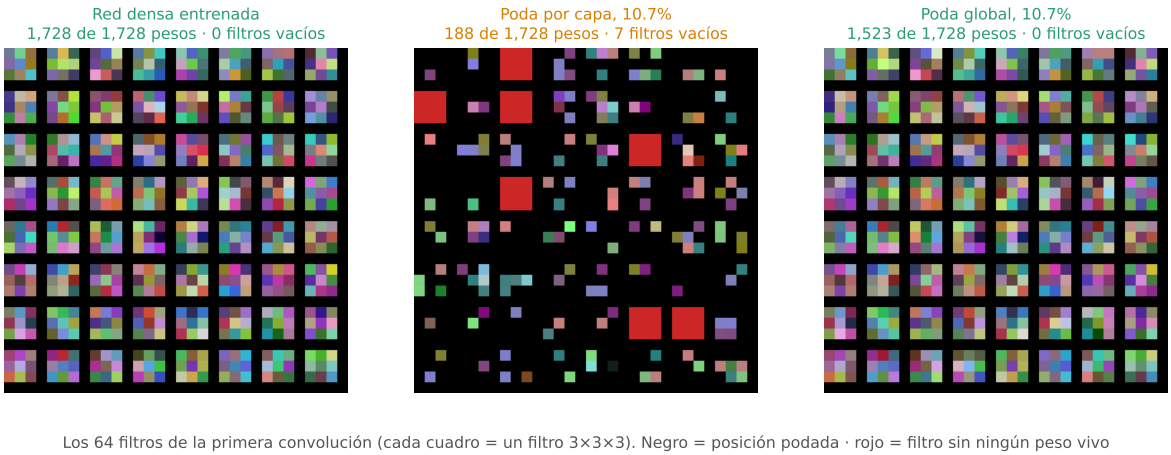


Figura 7: Los 64 filtros de la primera convolución de la red densa, del ticket con poda por capa y del ticket con poda global (10.7 % de los pesos). Cada cuadro es un filtro $3 \times 3 \times 3$ (el color son los tres canales). Negro: posición podada. Rojo: filtro sin ningún peso vivo.

Cómo leerlas: un filtro convolucional es un detector (de bordes, de colores) que se aplica a toda la imagen; si todos sus pesos se podan (cuadro rojo), su salida es constante y el detector deja de aportar. Con poda por capa conv1 queda con 188 de 1,728 pesos y 7 de sus 64 filtros están vacíos; con poda global conserva 1,523 y ninguno está vacío, y sus filtros se parecen a los de la red densa.

	Por capa (02)			Global (02b)		
Capa	% vivo	Pesos	Vacíos	% vivo	Pesos	Vacíos
conv1 (3→64)	10.9	188	7/64	88.1	1 523	0/64
conv2 (64→64)	10.7	3 960	0/64	51.0	18 794	0/64
conv3 (64→128)	10.7	7 918	0/128	44.4	32 700	0/128
conv4 (128→128)	10.7	15 834	0/128	38.2	56 283	0/128
fc1 (8192→256)	10.7	225 181	–	4.8	101 026	–
fc2 (256→256)	10.7	7 038	–	73.8	48 360	–
fc3 (256→10)	10.8	276	102/256	66.4	1 701	0/256

Cuadro 7: CNN, ronda 10 (10.7 % de los pesos). "Vacíos": filtros convolucionales sin ningún peso vivo; en fc3, neuronas ocultas sin ningún peso hacia la salida.

Las máscaras permiten ver el mecanismo (Tabla 7): con poda por capa la primera convolución queda con 188 de 1,728 pesos, 7 de sus 64 filtros están vacíos y 102 de las 256 neuronas ocultas no llegan a la salida. Con poda global la primera convolución conserva el 88 % de sus pesos, no hay filtros ni neuronas vacíos, y la primera capa lineal, que concentra el 86 % de los pesos y es la más redundante, absorbe la poda (queda en 4.8 %). Esto es consistente con H3 y con el uso de poda global en las redes profundas de [1].

Limitaciones de esta conclusión. (1) Es una sola corrida por configuración, y 02 y 02b tienen inicializaciones distintas. (2) 02b cambia *dos cosas a la vez* desde el punto de vista del mecanismo: el alcance de la poda y, en consecuencia, cuántos pesos conserva cada capa; el experimento no separa "la poda global" de "que cada capa conserve más". Se separaría con poda por capa que fije un mínimo

de pesos por capa. (3) El dense baseline también cambió (73.0 % en 02, 74.0 % en 02b), lo que da una idea del ruido entre corridas (∼1 punto).

¿Por qué la poda global conserva las capas pequeñas? Tamaño de los pesos. La poda global elimina los pesos de menor magnitud de toda la red, así que lo que sobrevive en cada capa depende de qué tan grandes son sus pesos. En la CNN densa entrenada la mediana de $|w|$ es de 0.105 en conv1 (1,728 pesos) y de 0.0086 en fc1 (2.1M pesos), unas 12 veces menos; fc3 (2,560 pesos) tiene 0.064. Un umbral global tira primero los pesos diminutos de fc1 y casi no toca a conv1 y fc3 (Figura 8, paneles izquierdo y central). La poda por capa ignora esa diferencia y obliga a conv1 a perder tantos pesos como fc1, lo que deja filtros y neuronas vacíos. Las diferencias de magnitud son coherentes con la inicialización por defecto de PyTorch, cuya escala es $\sim 1/\sqrt{\text{fan-in}}$: conv1 tiene 27 entradas por neurona y fc1 tiene 8,192, así que cada peso de conv1 carga más información y empieza más grande. En el ViT la relación es parecida pero menos limpia (Figura 8, panel derecho): `head` (37 % vivo) y `patch_embed` (25 %) tienen los pesos más grandes y sobreviven más que `qkv`, `mlp.0`, `proj` y `mlp.2`.

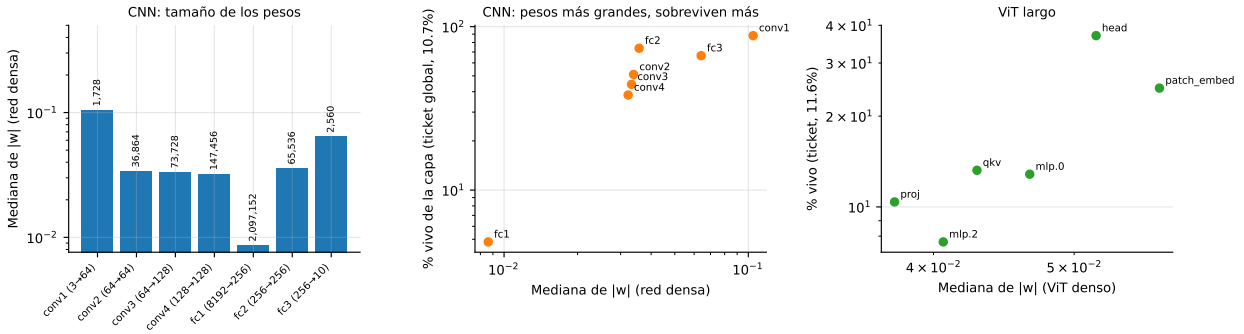


Figura 8: Tamaño de los pesos por capa (mediana de $|w|$ en la red densa) y fracción que sobrevive a la poda global. Izquierda: CNN, con el número de pesos de cada capa sobre las barras. Centro: CNN, mediana de $|w|$ contra % vivo con 10.7 % de los pesos totales. Derecha: ViT largo con 11.6 %.

Qué se puede y qué no se puede concluir sobre redes más grandes (H3b). Con la CNN se puede concluir que la poda por capa fue la causa del colapso y que el mecanismo es el cuello de botella en las capas pequeñas: son las capas con los pesos más grandes y se les obligó a perder la misma fracción que a la capa enorme y redundante. Eso *no* prueba que toda red grande necesite poda global: (1) ResNet y ViT no se podaron por capa, así que no hay comparación en ellas; (2) el MLP se podó por capa sin problemas (97.2 % con 3.5 % de los pesos) aunque su capa de salida tiene solo 1,000 pesos, así que la poda por capa no falla siempre; (3) la CNN es una red con una capa que concentra el 86 % de los pesos, que es el caso más favorable para la poda global. Lo que sí es razonable esperar es que el problema crezca con la heterogeneidad de tamaños: el ViT tiene capas desde 1,920 pesos (`head`) hasta 442,368 (`mlp`), y Frankle y Carbin [1] usan poda global justamente en las redes profundas. La prueba directa sería repetir el ViT (04, unos 45 min de GPU) y la ResNet (03, unos 1 h 45 min) con poda por capa.

6.6. Inspección de los tickets del ViT largo

Se inspeccionaron los checkpoints de 05 a 11.6 % de los pesos (ronda 5). Las figuras de esta sección se generan con `report/make_figures.py`; `06_inspection` contiene vistas adicionales (cabezas de atención, IoU entre ticket y rewinding, forma de las máscaras).

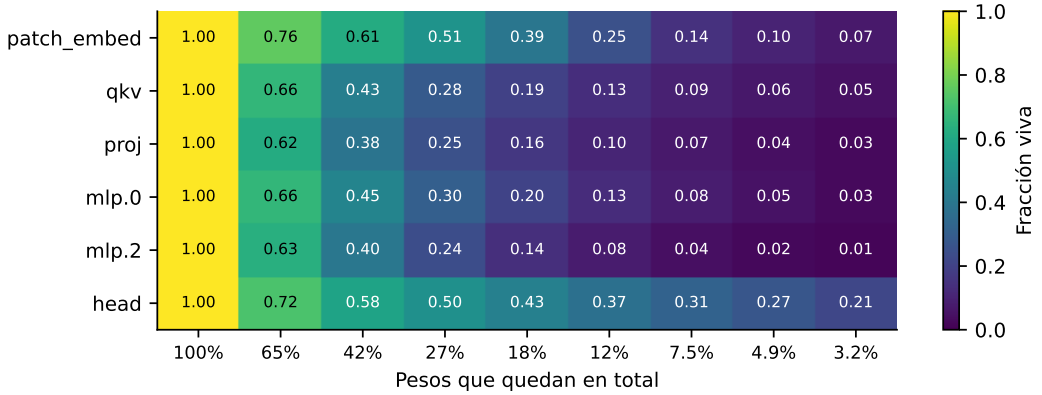


Figura 9: ViT largo: fracción viva por tipo de capa y ronda de poda (promedio ponderado sobre los 6 bloques).

Pesos vivos por capa. *Cómo leerla:* clara = casi todos los pesos vivos; oscura = casi vacía. Con poda global las capas no pierden parejo: en la última ronda (3.2 % del total) `head` conserva 21 %, `patch_embed` 7 %, `qkv` 4.5 %, `proj` 2.6 %, `mlp.0` 2.7 % y `mlp.2` 1.3 %.

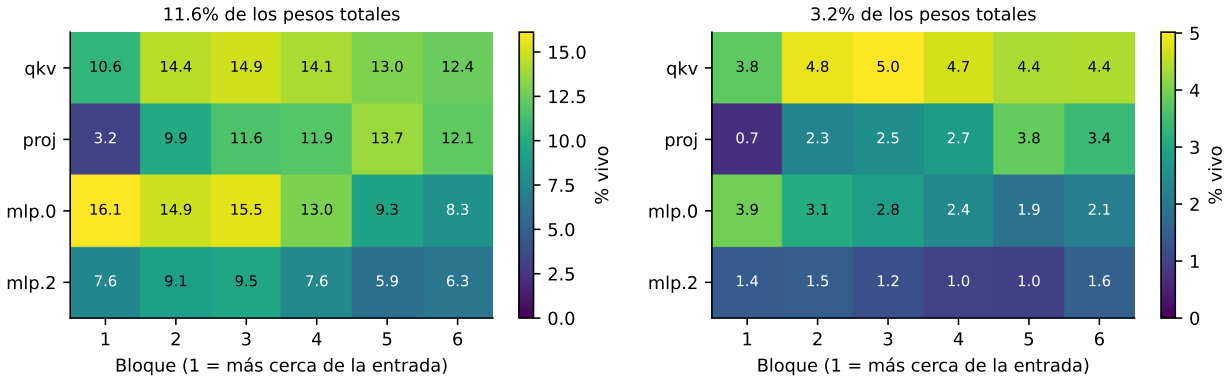


Figura 10: ViT largo: porcentaje de pesos vivos de cada capa dentro de cada bloque (1 = el más cercano a la entrada), con 11.6 % y 3.2 % de los pesos totales.

Cómo leerla: cada celda es una capa de un bloque; clara = más pesos vivos. Los patrones son: `mlp.0` pierde más pesos a mayor profundidad (16.1 % en el bloque 1 a 8.3 % en el 6 con 11.6 % de los pesos), `proj` hace lo contrario (3.2 % a 12.1 %), y `qkv` es casi plano. La capa `proj` del primer bloque queda casi vacía (0.7 % con 3.2 % de los pesos). No se probó por qué; una posibilidad es que la proyección de la primera capa de atención aporte poco porque las representaciones aún son simples, pero con una corrida y sin ablación es solo una hipótesis.

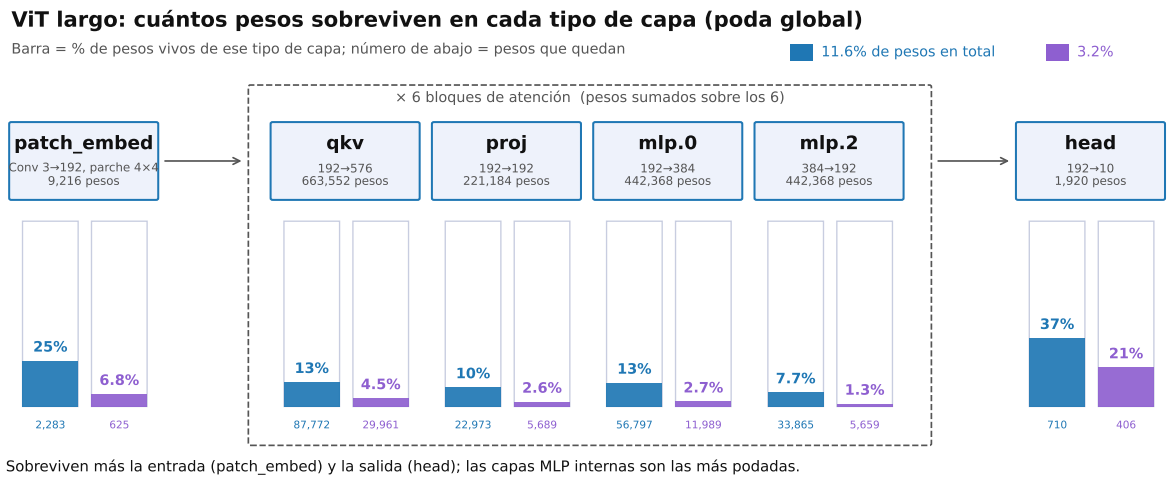


Figura 11: ViT largo: fracción de pesos vivos por tipo de capa (sumados sobre los 6 bloques) con 11.6 % y con 3.2 % de los pesos totales, y número de pesos que quedan.

Cómo leerla: es la misma información que el mapa de calor, ordenada como la red: entrada (`patch_embed`), bloques de atención y salida (`head`). Las dos capas extremas conservan más; dentro de los bloques, `mlp.2` es la más podada.

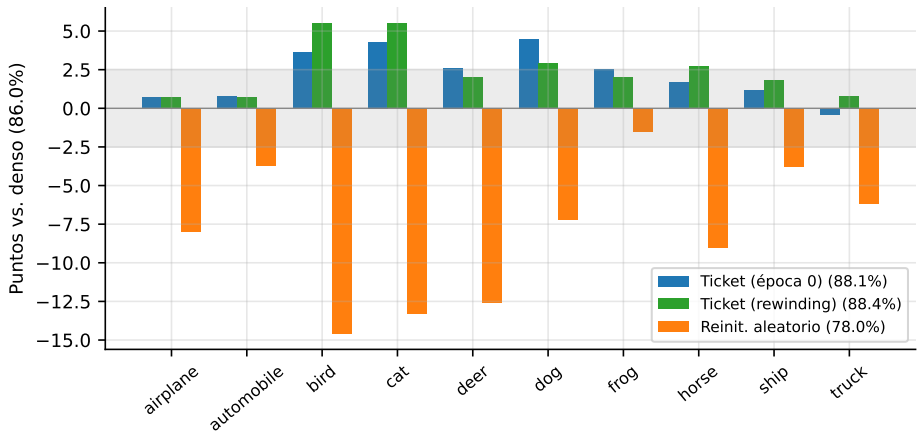


Figura 12: ViT largo: diferencia de accuracy por clase respecto al denso (11.6 % de los pesos). Banda gris: ruido de ±2,5 puntos.

Accuracy por clase. *Cómo leerla:* barras positivas = el modelo podado acierta más que el denso en esa clase; solo lo que cae fuera de la banda gris se interpreta (cada clase tiene 1,000 imágenes). El ticket queda entre −0,4 y +4,5 puntos del denso (mejor: perro), el rewinding entre +0,7 y +5,5 (mejor: gato), y el control aleatorio pierde en todas las clases, hasta 14.6 puntos en pájaro.

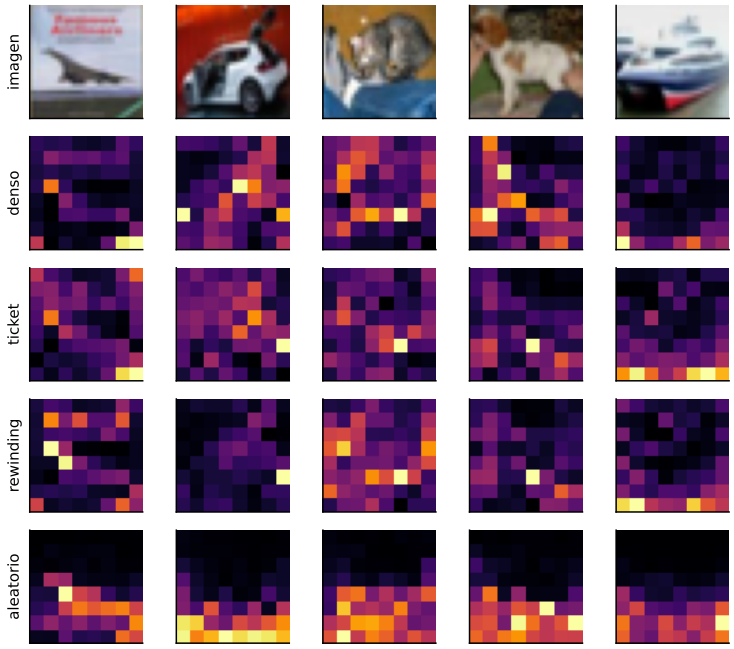


Figura 13: ViT largo: atención del `[CLS]` en el último bloque (promedio de cabezas) para 5 imágenes de prueba (columnas); cada fila es un modelo.

Mapas de atención. *Cómo leerla:* cada cuadro de la matriz 8×8 es un parche de 4×4 píxeles; más brillante = más atención del `[CLS]`, el token que decide la clase. *Cautela importante:* esto describe *dónde pone peso la atención*, no *qué usa el modelo para decidir*; la atención no es una explicación (Jain y Wallace, 2019) y aquí se midió: su correlación con la oclusión es ≈ 0 (Figura 15). Los mapas son visiblemente distintos entre modelos, pero no se concluye nada causal de eso.

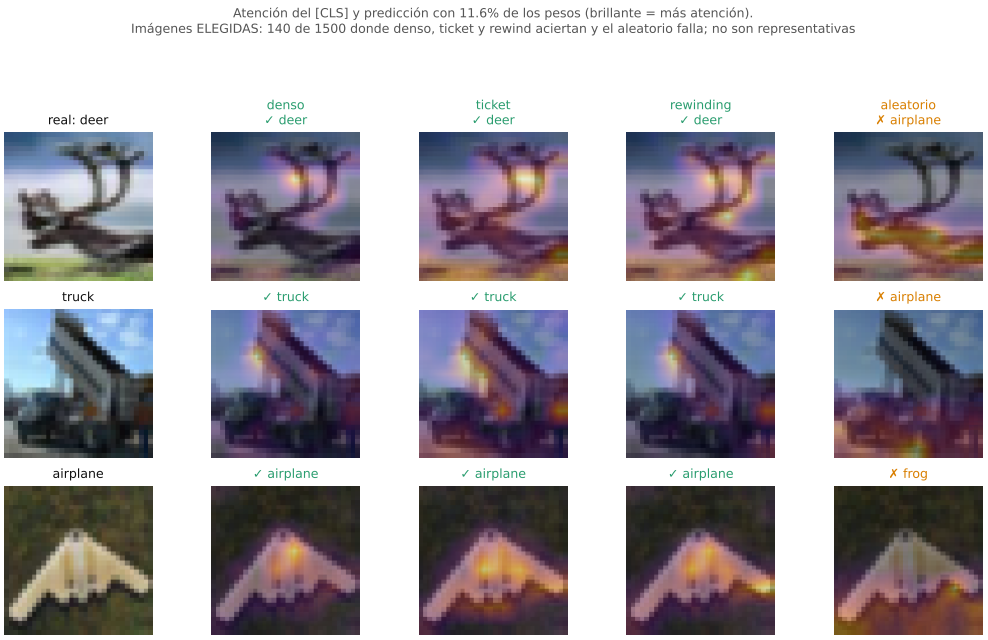


Figura 14: Atención del `[CLS]` sobre tres imágenes de prueba, con predicción de cada modelo (11.6 % de los pesos). **Las imágenes están elegidas**: son 3 de las 140 (entre las primeras 1,500 de prueba) en las que denso, ticket y rewinding aciertan y el control aleatorio falla; no son representativas del conjunto de prueba.

Cómo leerla y qué no se puede concluir: brillante = el `[CLS]` mira más ese parche. En estas imágenes los mapas del control (misma máscara que el ticket, otra inicialización) se ven distintos y apuntan hacia abajo o el fondo, y el control falla. Como el criterio de selección fue justamente que el control falle, la figura solo ilustra los mapas; no mide qué tan frecuente es ni sirve para explicar el fallo, como muestra la figura siguiente. La medición de qué tan bien funcionan los modelos es la accuracy por clase y la global (Tabla 5).

¿La atención explica? Atención contra oclusión. Para comprobar si el mapa de atención dice qué partes de la imagen usa el modelo, se compara con una medida causal simple: tapar cada bloque de la imagen con su color medio y medir cuánto baja la probabilidad de la clase correcta (oclusión). Si la atención explicara la decisión, los bloques con más atención deberían ser los que más probabilidad quitan al taparse.

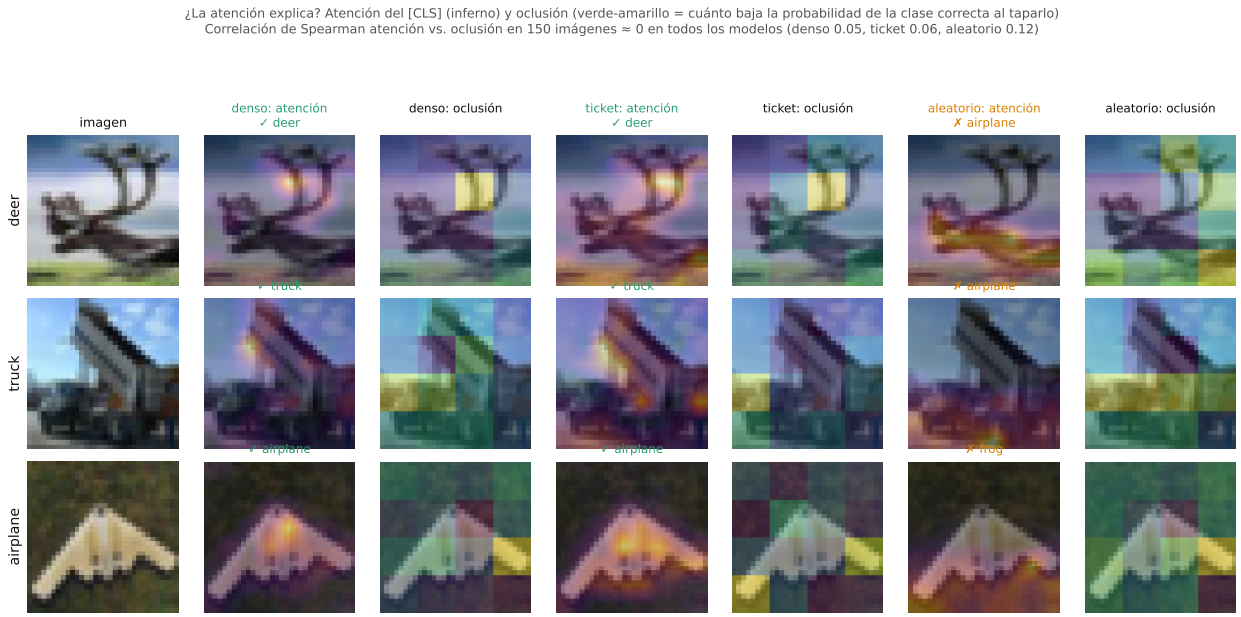


Figura 15: Atención del `[CLS]` y oclusión de bloques de 8×8 píxeles (amarillo = al taparlo baja más la probabilidad de la clase correcta), para el denso, el ticket y el control aleatorio, con 11.6 % de los pesos. Mismas 3 imágenes elegidas de la figura anterior.

Resultado. La correlación de Spearman entre atención y oclusión, promediada sobre las primeras 150 imágenes de prueba, es ≈ 0 en los cuatro modelos: 0.05 (denso), 0.06 (ticket), 0.03 (rewinding) y 0.12 (aleatorio) con bloques de 8×8 (con parches de 4×4: 0.01, 0.03, 0.01, 0.07). Usar *attention rollout* en lugar de la atención del último bloque tampoco ayuda (0.04–0.09). En el avión, por ejemplo, la atención se concentra en el ala pero la oclusión indica que el modelo depende más de la esquina inferior derecha. **Conclusión: en estos modelos la atención cruda del `[CLS]` no predice qué partes de la imagen necesitan, así que los mapas de atención se presentan como descripción y no como explicación.** Limitaciones: 150 imágenes, oclusión con color medio (puede introducir artefactos) y una sola corrida; los números salen de `results/attention_vs_occlusion.json`.

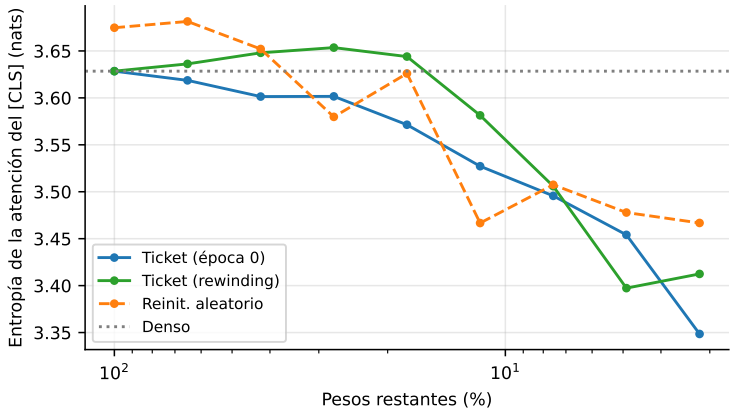


Figura 16: ViT largo: entropía de la atención del `[CLS]` frente al porcentaje de pesos restantes.

Entropía de la atención. *Cómo leerla:* máximo teórico $\ln 65 \approx 4{,}17$ (atención uniforme); cercano a 0 = una sola llave. Todos los modelos están lejos del máximo ($\approx 3{,}6$ el denso) y la entropía baja al podar, es decir, la atención se concentra en menos parches; no dice si esa atención es correcta.

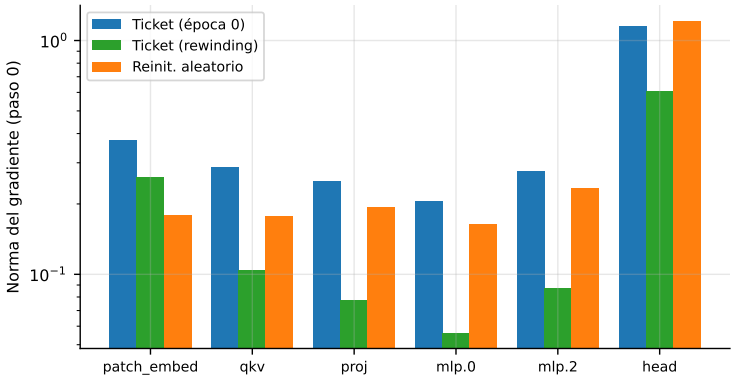


Figura 17: ViT largo: norma del gradiente sobre los pesos vivos en el paso 0, por tipo de capa (escala logarítmica).

Gradientes en el paso 0. *Cómo leerla:* una barra el doble de alta es un gradiente dos veces mayor; se compara la misma capa entre modelos, no capas distintas entre sí. El rewinding arranca con normas menores en las capas internas que el ticket (por ejemplo `qkv`: 0.10 frente a 0.29) y el control aleatorio queda cerca del ticket; en `head` las normas son 1.16 (ticket), 0.61 (rewinding) y 1.21 (aleatorio).

Borrador de interpretación (editar)

`head` y `patch_embed` conservan más pesos. Con poda global se elimina el 35 % de menor magnitud *de toda la red*, así que sobreviven más las capas con pesos grandes. `head` y `patch_embed` son pequeñas y están en los extremos de la red, donde no hay la redundancia de los seis bloques del medio; es el mismo patrón que en la CNN global, donde la primera convolución conserva 88 %. Es una explicación plausible, no probada; se confirmaría mirando la distribución de magnitudes por capa. **Atención del control aleatorio abajo.** Los mapas del control (misma máscara, otra inicialización) son distintos a los del ticket, pero como la atención no se correlaciona con la oclusión no se puede decir que el control falle *porque* mira otra zona; solo se puede decir que la inicialización cambia los mapas. **Rewinding con gradientes menores.** Desde la época 5 los pesos ya salieron del arranque más abrupto; es coherente con que sea el final del warmup, pero no se midió si eso *causa* la mejora a baja densidad. **Clases.** El ticket queda dentro del ruido en la mayoría de las clases, así que la ventaja no viene de una sola clase, aunque con 1,000 imágenes por clase tampoco se puede afirmar que sea pareja.

6.7. Incertidumbre

Una sola semilla por experimento y 10,000 imágenes de prueba. El intervalo binomial del 95 % sobre una accuracy de ∼88 % es de ±0,6 puntos; las diferencias entre ticket y control (5–9 puntos), entre ticket y denso a 27.5 % de los pesos (+3 puntos) y el colapso de la CNN (−8 puntos) lo superan; ticket frente a denso con 7.5 % de los pesos no. Este intervalo mide solo el ruido del conjunto de prueba, no la variación entre semillas, que no se midió y puede ser mayor (por ejemplo, el denso de la CNN dio 73.0 % y 74.0 % en dos corridas).

7. Discusión

7.1. Qué dicen los resultados sobre cada hipótesis

Hipótesis	Veredicto	Base
H0 existencia	**apoyada**	ticket $\geq$ denso y $>$ aleatorio en las cuatro arquitecturas (ResNet: con rewind; CNN: con poda global en todo el rango)
H1 complejidad → rewind	**parcial**	ResNet lo necesita, MLP y CNN no, ViT solo a poca densidad; no hay barrido de k y la relación no es monótona
H2 warmup ≈ rewind	**compatible, no probada**	patrón observado entre ResNet y ViT, confundido con optimizador, normalización y datos
H3a global > por capa (CNN)	**apoyada**	53.0 % → 75.7 % a 10.7 %; máscaras sin filtros ni neuronas vacíos; los pesos de las capas pequeñas son mucho mayores (Fig. 8); una sola corrida
H3b redes grandes necesitan global	**consistente, no probada**	solo la CNN tiene ambas variantes; el MLP funciona por capa; ResNet y ViT nunca se podaron por capa

Cuadro 8: Estado de las hipótesis.

Sobre "más compleja la red, más tarde aparece el ticket". Los datos son compatibles de forma cualitativa, pero no son una prueba: cada arquitectura tiene como mucho uno o dos puntos de rewind, y las arquitecturas difieren en más que su complejidad (datos MNIST frente a CIFAR-10,

optimizadores y *schedules*, alcance de la poda, tasas de poda, número de épocas).

Sobre el alcance de la poda. El resultado de la CNN muestra que la forma de podar puede cambiar la conclusión más que la arquitectura. Eso afecta la comparación entre experimentos: el MLP y la primera CNN podaron por capa y la ResNet y el ViT globalmente, así que parte de lo que parece diferencia entre arquitecturas puede ser diferencia entre tipos de poda.

7.2. Limitaciones

- Una sola semilla por experimento, sin barras de error ni pruebas de significancia.
- No se barrió el punto de rewind k; no se puede decir *cuándo* se fija el ticket en cada arquitectura.
- El warmup no se manipuló (H2).
- Presupuesto de cómputo: el ViT largo tarda ∼5 horas por corrida; un barrido de k sobre cuatro arquitecturas con varias semillas queda fuera del alcance.
- Desviaciones respecto a las fuentes (épocas por ronda, tasas de poda, *learning rate* del ViT sin ajustar): Apéndice A.
- No se probó si un ticket encontrado en un dataset sirve en otro.

7.3. Preguntas de discusión generadas con IA

Generado con IA

Las preguntas de esta subsección y sus borradores de respuesta fueron generadas por una IA (Claude, mediante Claude Code) a partir de los resultados del proyecto. No forman parte del análisis ni de las hipótesis del autor (Secciones 1–6); se incluyen como material de discusión, para mostrar qué preguntas puede plantear una IA sobre estos datos y contrastarlas con las conclusiones propias. Cada una indica qué experimento la resolvería; las respuestas solo usan lo que los datos permiten decir.

P1. *La ResNet necesita rewinding con lr 0.1 y sin warmup; el ViT con warmup no lo necesitó a 11.6 % y solo lo aprovechó por debajo de ∼8 %. ¿El factor es la arquitectura o la estabilidad del arranque?* **Experimento:** ResNet con warmup, ViT sin él. **Borrador:** los datos no separan ambas cosas; el patrón es compatible con que el warmup haga el trabajo del rewinding, pero también con que el ViT sea más tolerante. Confundidos: optimizador, datos, tasa de poda.

P2. *En el ViT largo el rewinding no ayudó con poco podado pero sí con mucho. ¿El punto de rewind necesario depende del nivel de poda?* **Experimento:** barrido de k a dos o tres sparsidades fijas. **Borrador:** los datos lo sugieren (sin efecto a ≥12 %, +3.4 puntos a 7.5 %), pero hay un solo punto de rewind y una semilla: una pista, no una conclusión.

P3. *¿1 época es "tarde" en una ResNet y 5 épocas en un ViT, o conviene medirlo en fracción del entrenamiento?* **Borrador:** como fracción, 1 de 77 épocas (1.3 %) frente a 5 de 100 (5 %); en iteraciones, 391 frente a ∼1,950. No hay unidad natural; lo razonable es reportar las tres y barrer k.

P4. *El control aleatorio (misma máscara, pesos nuevos) falla por mucho. ¿Qué parte la aportan los valores exactos y qué parte la escala?* **Experimento:** reinicializar permutando los valores del ticket dentro de cada capa. **Borrador:** la estructura sola no basta (el control pierde 5–9 puntos en el ViT largo y ∼2 en el MLP); falta separar valores de escala.

P5. *En el ViT largo el ticket supera a la red densa con ∼12 % de los pesos. ¿Es regularización por poda o un baseline mejorable?* **Experimento:** más épocas o más aumento al denso; varias semillas. **Borrador:** el denso de 100 épocas llega a 86.0 % y el ticket a 89.0 % con 27.5 % de los pesos, así que no es solo un baseline débil; la hipótesis restante es que la poda actúa como regularizador.

P6. *[5] dice que estos tickets son difíciles en ViTs. ¿Qué condiciones del recipe explican la diferencia?* **Borrador:** [3] reporta DeiT-Small en ImageNet; aquí el ViT es pequeño (1.8M pesos), con aumento fuerte y poda global, en CIFAR-10. No se pueden separar esos factores con los datos; presentarlo con reserva.

P7. *El colapso de la CNN a 10.7 % coincide con filtros y neuronas vacíos. ¿Sería evitable con poda global?* **Respondida con 02b:** sí, el ticket pasa de 53.0 % a 75.7 % sin filtros ni neuronas vacíos. Queda abierto si es la poda global en sí o que la primera convolución conserve más pesos; se separaría fijando un mínimo por capa.

P8. *¿Una capa con 7 de 64 filtros vacíos es un problema de capacidad o solo desperdicio de pesos?* **Borrador:** es capacidad perdida (esas salidas son constantes), y se suma a que 102 de 256 neuronas ocultas no llegan a la salida; con 10.7 % de los pesos esa red perdía capacidad real. Probar un mínimo de pesos por filtro lo confirmaría.

P9. *¿Se puede usar la atención del control aleatorio para explicar por qué falla, si comparte la máscara con el ticket?* **Experimento:** atención contra oclusión (hecho, Sección 6.6). **Borrador:** no con estos datos: los mapas del control son distintos, pero la atención cruda tiene correlación ≈ 0 con la oclusión en todos los modelos, así que no indica qué partes de la imagen usan. Lo que sí se puede decir es que la inicialización cambia los mapas.

P10. *¿Qué tanto se parecen las máscaras del ticket y del rewinding?* **Experimento:** IoU de `06_inspection` contra el IoU esperado entre dos máscaras aleatorias de la misma densidad d, que es $d/(2-d)$. **Borrador:** si es claramente mayor, ambos métodos redescubren parte de la misma estructura; si es bajo y ambos funcionan, hay muchos tickets distintos.

P11. *Con una sola semilla y ∼10,000 imágenes de prueba, ¿qué diferencias son creíbles?* **Borrador:** ver Sección 6.7. Ticket frente a aleatorio, ticket frente a denso a 27.5 % y el colapso de la CNN superan el intervalo binomial; ticket frente a denso a 7.5 % no, y el ruido entre semillas no se midió.

P12. *¿Cómo afecta que la poda sea global en unos modelos y por capa en otros a la comparación entre arquitecturas?* **Borrador:** no es un detalle (el resultado de la CNN cambió). Lo limpio sería repetir cada arquitectura con ambos tipos de poda.

8. Conclusión

1. **Existen tickets en las cuatro arquitecturas.** En el MLP, la CNN y el ViT el ticket desde θ_0 iguala o supera a la red densa y al control; en la ResNet solo con rewinding a la época 1. El ViT de 100 épocas lo confirma con un baseline mejor (86.0 %).
2. **La necesidad de rebobinar depende de la arquitectura *y* del recipe de entrenamiento**, y con los datos actuales no se puede separar un efecto del otro. Es compatible con que el warmup cumpla el papel del rewinding (H2), pero esa hipótesis solo se observó.
3. **La forma de podar cambia el resultado.** Con poda por capa, el ticket de la CNN colapsa con 10.7 % de los pesos; con poda global no (75.7 % frente a 53.0 %). Las máscaras y el tamaño

de los pesos muestran por qué: las capas pequeñas (conv1, fc3) tienen los pesos más grandes y la poda por capa las vacía al exigirles perder lo mismo que la capa enorme fc1. Que las redes más grandes requieran poda global es consistente con esto, pero solo se comparó en la CNN.

4. **El ViT es donde más se profundizó:** el ticket supera a la red densa hasta $\sim$12 % de los pesos, el rewinding ayuda solo con poca densidad, y la inspección muestra que `head` y `patch_embed` conservan más pesos y que la atención cruda del `[CLS]` no es un buen explicador de qué parches usa el modelo (correlación ≈ 0 con oclusión).
5. **Alcance y honestidad de los datos:** una semilla por experimento; diferencias de $\sim$1 punto o menos son ruido.

8.1. Trabajo futuro

- Probar H2 de forma directa: ResNet con warmup y ViT sin warmup.
- Barrer el punto de rewind k en las cuatro arquitecturas, con varias semillas y pruebas de significancia (McNemar sobre las predicciones guardadas es la más barata).
- Probar H3b: repetir ViT y ResNet con poda por capa; y separar el efecto del alcance de la poda de la cantidad de pesos por capa (poda por capa con un mínimo por capa).
- Control aleatorio con valores permutados; transferencia de tickets entre datasets (SVHN).

Referencias

[1] Jonathan Frankle y Michael Carbin. «The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks». En: *International Conference on Learning Representations (ICLR)*. 2019. URL: `https://arxiv.org/abs/1803.03635`.

[2] Jonathan Frankle et al. *Stabilizing the Lottery Ticket Hypothesis*. arXiv preprint. 2019. URL: `https://arxiv.org/abs/1903.01611`.

[3] Bowen Shen et al. «Data Level Lottery Ticket Hypothesis for Vision Transformers». En: *International Joint Conference on Artificial Intelligence (IJCAI)*. 2023. URL: `https://arxiv.org/abs/2211.01484`.

[4] Alex Renda, Jonathan Frankle y Michael Carbin. «Comparing Rewinding and Fine-tuning in Neural Network Pruning». En: *International Conference on Learning Representations (ICLR)*. 2020. URL: `https://arxiv.org/abs/2003.02389`.

[5] Hugo Touvron et al. «Training data-efficient image transformers & distillation through attention». En: *International Conference on Machine Learning (ICML)*. 2021. URL: `https://arxiv.org/abs/2012.12877`.

A. Desviaciones respecto a las fuentes

Elemento	Fuente	Este proyecto
Épocas por ronda, MLP	50K iteraciones [1]	5 épocas (∼5K iteraciones)
Tasa de poda, MLP	10 % en la capa de salida	20 % en todas
Tasas de poda, CNN	conv 10 % / lineales 20 %	20 % en todas
Batch, CNN	60	64 (30 épocas)
Validación, ResNet	45K train / 5K val	50K train, sin validación
Punto de rewind, ResNet	iteración 500 [2]	época 1 = iteración 391
Normalización, ResNet	resta de media por píxel	media y desviación por canal
Estado del optimizador al rebobinar	continuo	se reinicia en la época de rewind
Tasa de poda, ViT	20 %	35 % (menos rondas para llegar a ∼3 %)
Learning rate, ViT	$5 \cdot 10^{-4} \times$ batch/512 $= 1{,}25 \cdot 10^{-4}$ [5]	10^{-3} (sin ajustar)
Warmup, ViT	5 épocas	2 de 20 (04) y 5 de 100 (05)
Semillas	promedio de varias corridas	una sola

Cuadro 9: Desviaciones respecto a las fuentes (detalle en `REFERENCES.md`).

B. Cómo reproducir

```
cd notebooks && uv run python 02b_cnn_global.py   # un experimento
uv run python report/make_figures.py              # figuras y tablas del informe
cd report && latexmk -pdf informe.tex             # este documento
cd notebooks && uv run marimo run 08_demo.py      # demo interactiva
```